# Lecture 03: Python, Google Colab and Tabular Data

## NT120 – Introduction to AI for Computer Networks and Cybersecurity

Huynh Van Dang, Phan The Duy, Tran Tuan Dung, Nguyen Khanh Thuat

Faculty of Computer Networks and Communications
University of Information Technology, VNU–HCM

danghv@uit.edu.vn

Course page: link.uit.edu.vn/NT120

Lab notebook: link.uit.edu.vn/NT120-Wk3

# Outline


1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# Learning Outcomes

By the end of this lecture, students will be able to:

- Run and modify a notebook in Google Colab, and explain the notebook execution contract
- Explain why data quality and pipeline discipline decide whether an ML system works
- Describe a table of network flow records: instances, features, labels and feature types
- Use the Python constructs required for data work, and NumPy arrays instead of index loops
- Inspect a tabular dataset with Pandas: shape, types, distributions and missing values
- Select, filter, group and join tabular data, and check the result of each step
- Apply the reproducibility rules used throughout this course

**Course learning outcomes addressed:** G2.1

# Today's Lab Notebook



`link.uit.edu.vn/NT120-Wk3`

Scan the QR code or open the link. Before you type anything, choose *File* → *Save a copy in Drive*, so that your edits are kept.

- Sections 1 to 6 follow this lecture: Colab setup, Python, NumPy, Pandas, joins and a checkpoint
- Section 7 is the in-class activity: six questions on the course table, each with a self-check that says right or wrong without giving the answer
- Section 8 and the optional stretch tasks are for after class

Cells titled "Your turn" hold a TODO. The notebook always runs from top to bottom, even when it is unfinished, so *Runtime* → *Restart session and run all* is your final test before you submit.

# The 90 Minutes

| 0 to 10 min | 10 to 20 min | 20 to 55 min | 55 to 60 min | 60 to 82 min | 82 to 90 min |
|---|---|---|---|---|---|
| Recap | Setup | Live coding | Checkpoint | Activity | Wrap-up |
| homework answers | notebook §1 | notebook §2 to §5 | notebook §6 | notebook §7 | notebook §8 |

*Sections are numbered in the file. Every activity question has a self-check that says whether you are right, without revealing the answer.*

**Before the lab**

- Sign in to a Google account
- Open the notebook link, then save a copy to your Drive

**During the lab**

- Run cells top to bottom with Shift+Enter
- Type in the “Your turn” cells only

**After the lab**

- Restart and run all
- Submit the executed `.ipynb`

The theory in this deck is longer than one lecture slot. Slides marked “Reference”, and the appendix, are for self-study.

# Today's Dataset

Everything in Sessions 3 to 10 uses one running dataset, so that the effort goes into the method rather than into re-learning the columns.

- A teaching-sized table of network flow records, in CSV form, with the structure of a public intrusion detection corpus
- Each row is one **bidirectional network flow**; Session 4 explains where those rows come from
- 112 450 rows and 79 columns: numeric measurements, a categorical `protocol` column and a `label` column
- The label is binary today: `BENIGN` or `PortScan`
- The notebook generates the file with a fixed seed, so every student starts from the same table. The values are simulated: do not draw conclusions about real attack traffic from them

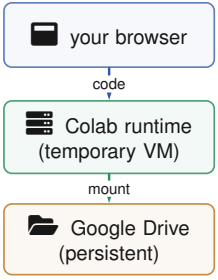
You do not need to understand the columns yet. Today's goal is to *look* at them: shape, types, ranges, missing values, class balance.

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# What Colab Actually Is

- A Jupyter notebook running on a **temporary virtual machine** that Google gives you
- Python, NumPy, Pandas, Matplotlib and Scikit-learn are pre-installed: nothing to set up
- The machine is discarded after a period of inactivity, and *everything you wrote to its disk goes with it*
- Files reach it by upload, by mounting Google Drive, or by downloading from a URL
- A CPU runtime is enough for this course; a GPU is optional and not needed today




*work that must survive the session is written to Drive, not to the runtime*

# Colab Idioms You Will Use

```
# a leading ! runs a shell command on the runtime
!pip list | grep -i pandas
!ls -lh /content

# a leading % is a "magic" for the notebook itself
%timeit np.arange(1_000_000).sum()

# get files onto the runtime (temporary)
from google.colab import files
uploaded = files.upload()

# persistent storage: mount your Drive
from google.colab import drive
drive.mount("/content/drive")
df.to_csv("/content/drive/MyDrive/nt120/out.csv",
          index=False)
```

## The Runtime menu

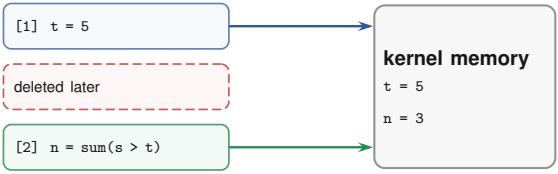
- *Restart session* clears every variable, keeps the text
- *Run all* executes the cells from the top
- *Change runtime type* selects CPU, GPU or TPU
- *Disconnect and delete runtime* discards the machine and its disk

Idle sessions are disconnected and every session has a maximum lifetime; Google does not guarantee either limit. Never rely on the runtime disk for work you cannot re-create.

# Cells, Kernel and Execution Order

- A notebook is a list of **cells**: code cells run Python, text cells hold Markdown
- One Python process, the **kernel**, runs every code cell and keeps the variables in memory
- The number in brackets, `[3]`, records the *order in which cells ran*, not their order on the page
- The saved `.ipynb` file stores your code and your outputs, but *not* the kernel's variables

Shift+Enter runs a cell and moves on; Ctrl+M B (Cmd+M B on macOS) adds a cell below.




*t stays in memory after its cell is deleted, until the session restarts, and then the last cell fails.*

# The Notebook Contract

**✖ What people do**

```
[1]  load the CSV
[7]  drop a column
[3]  plot something
[2]  drop it again, and get an error
```

*execution counters out of order: the result depends on the order you happened to click*

**✔✔ What we require**

```
[1]  imports and seed
[2]  load the CSV
[3]  clean, in code
[4]  inspect and plot
```

*counters in order: Restart and run all reproduces every number*

⚠

**Hidden state**

A variable can exist in memory long after you deleted the cell that made it. The notebook works for you and fails for everyone else.

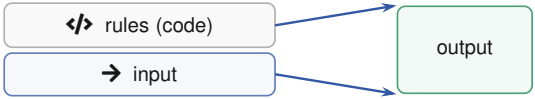
**Restart and run all** before every submission. If it fails, the notebook was never correct: you were just lucky about the order.

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

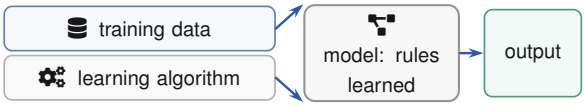
# Software Logic Versus Learned Behavior

**Traditional software**




**Bad input:** an exception or an error you can see

**Machine learning**




**Bad data:** a model that still answers, only quietly worse

- In ordinary software the logic is in the code. In a machine learning system the behavior is defined by the data, so data must be engineered with the same rigor as source code
- **Data engineering** is the discipline of designing, building and maintaining the infrastructure that turns raw information into reliable, accessible, ML-ready datasets: acquisition, processing, storage and governance

[MLS] Ch. 6, §6.1, pp. 324–325 (paraphrased)

# Data Problems Fail Silently

Three problems in today's dataset. None of them raises an error.

### Numbers stored as text

**What you see**

`bwd_bytes` holds `"6,520"`. `df.describe()` runs without complaint

**What is true**

The column is simply missing from the summary

### Missing values

**What you see**

`df["flow_bytes_s"].mean()` returns a number

**What is true**

It was computed on 112 298 rows: pandas skipped the 152 missing ones

### Class imbalance

**What you see**

A model that says `BENIGN` every time scores 99.63% accuracy

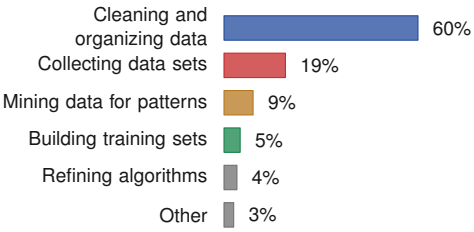
**What is true**

It detects none of the 416 port scans

An ML system does not crash on bad data. It keeps producing outputs and gets a little worse, and the problem is usually found after deployment. Your job in this course is to look before you model.

[MLS] Ch. 6, §6.1, p. 325: quality problems appear as gradual degradation, not as error messages

# Where the Time Goes




*Share of a data scientist's time, as summarized in Fig. 6.2 of [MLS]*

- Cleaning and organizing data takes about 60% of the effort. Building and refining models takes less than a tenth
- Industry surveys quoted in [MLS] put data preparation at 60 to 80% of a data scientist's time
- The same proportions will show up in your notebooks: most cells in today's lab inspect, fix and check data

"Garbage in, garbage out" is a 1960s saying and still the best summary. No algorithm rescues a table whose columns are mistyped, mislabeled or missing.

[MLS] §6.2.2, Fig. 6.2, p. 328, redrawn; the source is "various industry reports", so the shares are indicative

# Four Pillars of Data Engineering

**Quality**

Accurate, complete, consistent and fit for the ML task. Errors here compound downstream

In today's notebook: `info()`, `isna()`, `to_numeric`, the text column `bwd_bytes`

**Reliability**

Keeps working despite failures, anomalies and load; errors are handled and monitored

In today's notebook: Restart and run all; self-checks print TODO and never stop the notebook

**Scalability**

Handles growth in data volume and demand without a redesign

In today's notebook: vectorized expressions instead of index loops, so the code survives a bigger table

**Governance**

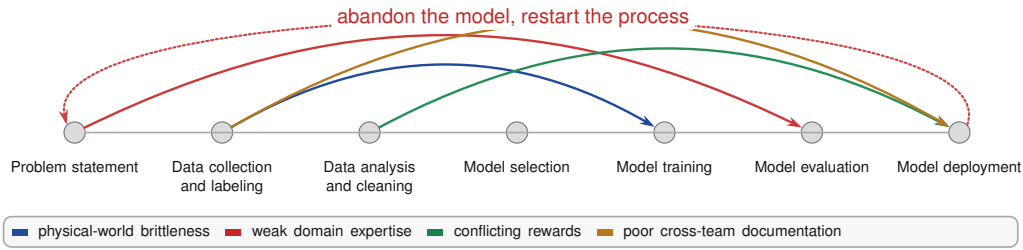
Privacy, bias, compliance, ownership and access control around the data

In today's notebook: simulated data, no real traffic; seed and library versions recorded

*The pillars pull against each other: validation slows throughput, strict consistency limits scale, privacy constrains performance, bias mitigation can shrink the data.*

[MLS] Ch. 6, §6.2.1–6.2.2, Fig. 6.1, pp. 326–327 (paraphrased)

# Data Cascades: Early Errors Compound




- A data cascade is a chain of downstream failures caused by poor data quality in an early stage. It is specific to ML systems
- Collection errors surface at evaluation or deployment, when the remedy may be to abandon the model and start again
- Prevention is cheap early: state quality criteria before collecting, and validate as data enters the pipeline

[MLS] §6.3, Fig. 6.3, p. 330, redrawn schematically after Sambasivan et al. (2021)

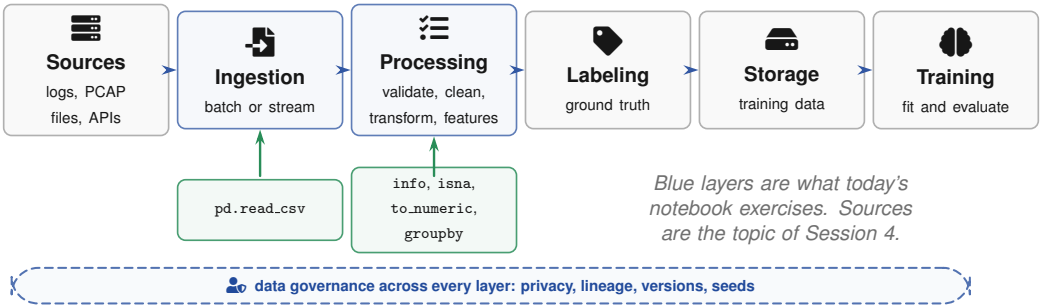
# What Quality Means for a Table

| Dimension | The question to ask | The Pandas call that asks it |
|---|---|---|
| Accuracy | Are values plausible, in the right units? | `describe()`, `min()`, `max()` |
| Completeness | How many values are missing, and where? | `isna().sum()`, `isna().mean()` |
| Consistency | One type and one format per column? | `info()`, `dtypes`, `value_counts()` |
| Uniqueness | Is any record counted twice? | `duplicated().sum()` |
| Timeliness | Is the data recent enough, in time order? | `min()`, `max()` of the timestamp |
| Fitness | Does the sample fit the task, with usable labels? | `value_counts(normalize=True)` |

Quality is not one number. Each dimension has its own check, and today's six-line inspection touches all of them.

[MLS] §6.2.1 and §6.7.1, pp. 326, 364. Uniqueness is added: duplicates are the textbook's first cleaning example

# Where a Notebook Sits in the ML Pipeline




- The notebook is not the pipeline, but it is where you prototype ingestion, validation and cleaning before they are frozen into code others will run
- Anything you do by hand in the notebook has to be repeated in serving; Section 9 returns to that requirement

[MLS] Ch. 6, §6.4, Fig. 6.5, p. 336: sources, ingestion, processing, labeling, storage, training, with governance across layers

# Four Pillars Across the Pipeline (Reference)

| | Quality | Reliability | Scalability | Governance |
|---|---|---|---|---|
| **Acquisition** | representative sampling, bias detection | diverse sources, redundant collection | web scraping, synthetic data | consent, anonymization, ethical sourcing |
| **Ingestion** | schema validation, data profiling | dead letter queues, graceful degradation | batch versus stream, autoscaling | access controls, audit logs, lineage |
| **Processing** | consistency checks, training-serving parity | idempotent transformations, retries | distributed frameworks, horizontal scaling | lineage tracking, privacy, bias monitoring |
| **Storage** | validation checks, freshness monitoring | backups, replication, recovery | tiered storage, partitioning, compression | access audits, encryption, retention |

Read the Quality column for ingestion and processing: profiling, schema validation, consistency checks. That is what Sections 6 to 8 of this deck do in Pandas.

[MLS] Ch. 6, §6.2.3, Table 6.1, p. 329 (condensed). Use it as a checklist when a pipeline misbehaves

# Checks a Production Pipeline Runs (Reference)

| Check | What [MLS] describes | Pandas version |
|---|---|---|
| Schema | types, ranges and required fields, checked as each record enters | `dtypes`, `between()` |
| Null rate | alert when nulls exceed 5% for a feature that had under 1% in training | `isna().mean()` |
| Count anomalies | a tenfold jump in records suggests duplication or a pipeline error | `len(df)`, `duplicated()` |
| Range violations | negative prices, impossible ages | `(df[c] < 0).sum()` |
| Category drift | new categories, or a large shift in category proportions | `value_counts(normalize=True)` |
| Numeric drift | rolling mean and spread, tested against the training data | `describe()` now, tests later |

The thresholds are the textbook's examples for a recommendation system. For network data you set your own, from the training table.

[MLS] Ch. 6, §6.4.1, pp. 337–338 and §6.7.1, p. 364 (paraphrased)

# A Validation Function for the Flow Table

```python
def validate(df):
    text = set(df.columns) - set(
        df.select_dtypes("number").columns)
    assert text == {"protocol", "label"}, \
        f"unexpected text columns: {sorted(text)}"
    assert set(df["label"]) <= {"BENIGN", "PortScan"}
    assert df["dst_port"].between(0, 65535).all()
    assert (df["duration"] >= 0).all()
    assert not df.duplicated().any()
    worst = df.isna().mean().idxmax()
    assert df[worst].isna().mean() < 0.01, worst
    return df

validate(pd.read_csv("flows.csv"))
```

**On the raw CSV it fails:**

```
AssertionError: unexpected text
columns: ['bwd_bytes', 'label',
'protocol']
```

The check found the mistyped column before any model did. After `.str.replace(",", "")` and `.astype("int64")`, it passes and returns the table.

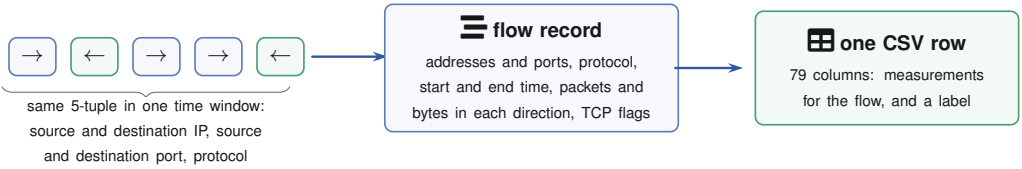
An `assert` turns a silent failure into a loud one. Write the checks once and call them after every load.

Sketch, not from the textbook: it applies the schema and null-rate checks of [MLS] §6.4.1

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# From Packets to a Flow Record




*Blue packets travel client to server (`fwd_*` columns); green packets travel back (`bwd_*` columns).*

- A **flow** groups the packets that share a 5-tuple within a time window; a **bidirectional** flow counts both directions
- NetFlow-style exporters record, for each flow, the addresses, ports, protocol, timestamps, TCP flags and the packet and byte counts
- Flow features use no payload, so they still work on encrypted traffic. Session 4 shows how flows are built from packet captures

[G&S] Ch. 1, §1 and §4.2 (paraphrased)

# Reading Three Rows

| | dst_port | protocol | duration | fwd_pkts | bwd_pkts | fwd_bytes | bwd_bytes | flow_bytes_s | label |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 53 | UDP | 3.395 | 6 | 5 | 527 | "6,520" | 2075.6 | BENIGN |
| 1 | 80 | TCP | 0.158 | 3 | 8 | 268 | "3,728" | 25219.3 | BENIGN |
| 10 | 381 | TCP | 0.016 | 2 | 0 | 0 | "0" | 0.0 | PortScan |

- Row 0 is a DNS lookup: a UDP exchange with about 3.4 s of activity and a large reply
- Row 10 is a probe: two packets from the client, nothing sent back, no payload in 16 ms
- Units matter: `duration` is in seconds, sizes in bytes, and the rate column is bytes per second

**A derived column**

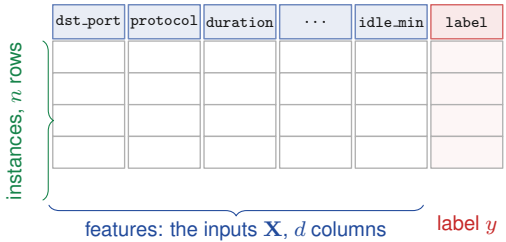
`flow_bytes_s = (fwd_bytes + bwd_bytes) / duration`

Row 0: (527 + 6520) / 3.395 = 2075.6

With `duration` = 0 there is nothing to divide by: 152 rows have no value

Note the quotation marks: `bwd_bytes` is stored as text with a thousands separator. It looks like a number and is not one.

# Instances, Features and Labels




- An **instance** is one row: here, one flow
- A **feature** is a column the model may use as input
- The **label** is the column the model must predict; it is never used as an input
- The table is a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ plus a vector $\mathbf{y}$

For the course table: $n = 112\,450$ flows, $d = 78$ feature columns, and $y \in \{\texttt{BENIGN}, \texttt{PortScan}\}$.

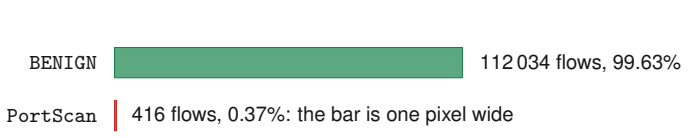
Supervised learning needs labeled instances. If the labels are wrong or missing, no model can be better than the labels.

# Feature Types in a Flow Table

| Type | Example column | dtype | What to watch for |
|---|---|---|---|
| Continuous | `duration`, `flow_bytes_s` | `float64` | heavy right tail; a mean of 18.3 s beside a median of 4.3 s |
| Count | `fwd_pkts`, `bwd_bytes` | `int64` | non-negative integers; a count of 0 is a real value, not a missing one |
| Flag or binary | `fwd_psh_flags` | `int64` | only 0 and 1; `nunique()` finds constant columns |
| Categorical | `protocol` | text | unordered; must be encoded before most models |
| Identifier-like number | `dst_port` | `int64` | stored as a number, meant as a category: port 443 is not “larger” than port 80 |
| Label | `label` | text | the target; check its balance first |

The dtype tells you how Pandas stored a column, not what it means. Deciding the meaning of each column is part of your job.

# The Label You Look at First: Class Balance




**always predict** `BENIGN`

| | |
|---|---|
| TN 112 034 | FP 0 |
| FN 416 | TP 0 |

*accuracy 99.63%, recall for* `PortScan` *0%*

- With this base rate, “accuracy” rewards a model that never raises an alarm. Report class counts beside every metric
- Networks are naturally imbalanced: attacks are rare next to normal traffic. Detectors trained on such data tend to label everything normal and still show accuracy close to 100%
- Two standard remedies: weight the loss so rare classes count more, and judge the model by precision, recall and F1 instead of accuracy

[G&S] Ch. 1, §5.2 (paraphrased). Session 1 introduced the accuracy paradox; Session 7 gives the metrics and Session 8 the remedies for imbalance

# Flow Features in the Literature

- One flow record carries few numbers, so detectors aggregate a **window** of recent flows into one feature vector
- Windows can be multi-scale, for example 5 minutes, 1 minute and 1 second
- The features depend on the threat: for TCP anomalies, compute TCP features
- The chapter computes 288 features per window and stresses **entropy** (how spread out addresses and ports are) and **variance** (how different the flows are)

| Family | Examples |
|---|---|
| Counts | flows in, flows out, share of symmetric flows |
| Volume | sum, max, min, mean, variance of packets and bytes |
| Diversity | distinct source IPs, distinct ports |
| Entropy | of source IPs, of destination ports |
| Port ranges | share of ports above 1024 |

The course table sits one step below this: one row per flow, with the per-flow statistics already computed. Window aggregates belong to the feature work that follows.

[G&S] Ch. 1, §4.2, Table 1 (paraphrased and condensed)

# Public Corpora: The CTU Botnet Dataset

- Captured at CTU University, Czech Republic, in 2011: 13 scenarios and seven botnet families
- Real botnet traffic, not simulation, mixed with unknown traffic from a large network, with ground-truth labels
- Each scenario is a packet capture (PCAP) that is post-processed into flow records
- The chapter builds feature vectors from 30 and 60 second views after every flow, and labels a vector anomalous when the last flow was

| Scenario | Packets | Size | Bot |
|---|---|---|---|
| 1 | 71 971 482 | 52 GB | Neris |
| 3 | 167 730 395 | 121 GB | Rbot |
| 5 | 4 481 167 | 37.6 GB | Virut |
| 8 | 155 207 799 | 123 GB | Murlo |
| 11 | 6 337 202 | 5.2 GB | Rbot |
| 13 | 50 888 256 | 34 GB | Virut |

Our teaching table has the shape of such a corpus (per-flow features plus a label) but it is simulated. It exists to teach the method, not to measure real attacks.

[G&S] Ch. 1, §5.1, Table 2 (selected rows, sizes as printed in the chapter)

# Relevant, Redundant and Irrelevant Features

- A feature is **relevant** if the decision depends on it, and **irrelevant** otherwise
- A feature is **redundant** if it is highly correlated with other features
- Removing redundant, irrelevant and misleading features can improve accuracy, and cuts the cost of training an expensive model
- Feature selection is a process with several stages; Session 5 returns to it. Today you only need to spot the candidates

## In the course table

- `subflow_fwd_pkts` equals `fwd_pkts` in every row
- `fwd_seg_size_avg` and `fwd_pkt_len_mean` have correlation 1.0
- `cwe_flag_cnt` and `bwd_urg_flags` never change, so they cannot help

```
df.nunique().sort_values().head()
df.select_dtypes("number").corr()
```

Before you ask a model to find structure, ask the table for the obvious structure: constant columns, exact copies and near copies.

[G&S] Ch. 8, §3 (paraphrased). The column facts were computed on the notebook's simulated table

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# The Subset You Actually Need

```
# lists, dicts, comprehensions
ports = [22, 80, 443, 3389]
names = {22: "ssh", 80: "http",
         443: "https"}

known = [p for p in ports if p in names]
labels = [names.get(p, "other") for p in
     ports]

# a function, not a repeated cell
def rate(bytes_, seconds):
    if seconds <= 0:
        return 0.0
    return bytes_ / seconds
```

- **Indentation is syntax**: the body of `def`, `if` and `for` is whatever is indented under it
- `names.get(p, "other")` returns a default instead of raising `KeyError`
- A function gives a repeated calculation one name, one place to fix a bug, and one place to test
- Guard the division: a duration of zero is normal in flow data

These twelve lines cover most of the Python you will write in this course. The rest is library calls.

# Containers and What They Are For

| Type | Literal | Property | Networking use |
|---|---|---|---|
| `list` | `[22, 80, 443]` | ordered, mutable | a column of ports before Pandas |
| `tuple` | `("10.0.0.5", 443, "TCP")` | ordered, immutable, hashable | a flow key |
| `dict` | `{22: "ssh", 80: "http"}` | key to value, fast lookup | port to service map |
| `set` | `{22, 80, 22}` | unique, unordered | distinct destination ports |
| `str` | `"192.168.0.1"` | immutable text | an address, before parsing |

```
counts = {}
for src, dst, port in flows:              # tuple unpacking
    key = (dst, port)                     # a tuple can be a dict key; a list cannot
    counts[key] = counts.get(key, 0) + 1
```

Pandas hides most of these loops, but you will read them in other people's code.

# Comprehensions, Functions and f-strings

```
sizes = [527, 6520, 268, 3728, 11409]

# loop
big = []
for b in sizes:
    if b > 1000:
        big.append(b / 1000)

# comprehension: the same result
big = [b / 1000 for b in sizes if b > 1000]
# [6.52, 3.728, 11.409]

# dict comprehension
total = sum(sizes)
share = {i: b / total for i, b in enumerate(
    sizes)}
```

```
def mbps(nbytes, seconds):
    """Megabits per second, 0.0 if seconds
     <= 0."""
    if seconds <= 0:
        return 0.0
    return 8 * nbytes / 1e6 / seconds

print(f"{mbps(6_520, 3.395):.3f} Mbit/s")
# 0.015 Mbit/s
```

- A comprehension reads as a sentence: "`b / 1000` for each `b` in `sizes` if `b > 1000`"
- `f"{x:.3f}"` formats a number in place; `6_520` is just `6520` with a readable separator
- The docstring is what `help(mbps)` prints

# Reading Error Messages

| The message | What it usually means | What to try |
|---|---|---|
| `KeyError: 'Duration'` | the column name is not spelled exactly so (case, spaces) | `df.columns` |
| `NameError: name 'df' is not defined` | the cell that creates `df` has not run in this session | Runtime, Run all |
| `ValueError: could not convert string to float` | a numeric column holds text, such as `"6,520"` | `df.dtypes`, `str.replace` |
| `TypeError: '>' not supported between 'str' and 'int'` | comparing text with a number | `df.info()` |
| `AttributeError: 'Series' object has no attribute ...` | a typo, or a method that belongs to another type | the spelling; `dir(df)` |

Read a traceback from the bottom: the last line names the error, and the line above it shows where it happened. Most of the fix is in those two lines.

# The Standard Header of Every Notebook

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

pd.set_option("display.max_columns", 40)
print(pd.__version__, np.__version__)
```

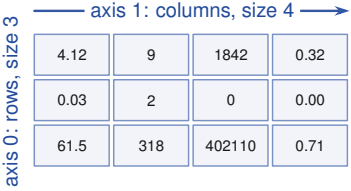
- `np`, `pd` and `plt` are conventional aliases; every tutorial and every colleague uses them
- One named constant for the seed, so it is changed in one place and printed in reports
- `display.max_columns` lets a 79-column table show all its columns
- The version line costs nothing and answers “which Pandas was this?” months later

This cell is always cell one. If a notebook does not start with it, a reader cannot tell what environment produced the results.

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# An Array Is Not a List




**shape** `(3, 4)`: the tuple of dimension sizes

**dtype** `float64`: *one* type for the whole array

**vectorized**: `a / a.sum(axis=0)` touches every cell with no Python loop

*A Python list can hold anything and is slow. A NumPy array holds one type in a fixed shape, and that restriction is exactly what makes it fast.*

# Creating and Inspecting Arrays

```
a = np.array([[4.12, 9, 1842, 0.32],
              [0.03, 2,    0, 0.00],
              [61.5, 318, 402110, 0.71]])

a.shape, a.ndim, a.dtype    # (3, 4)  2  float64
a[0, 2]                     # 1842.0   row 0, column 2
a[:, 1]                     # [9., 2., 318.]  one
    column
a[a[:, 0] > 1]              # rows where column 0 > 1

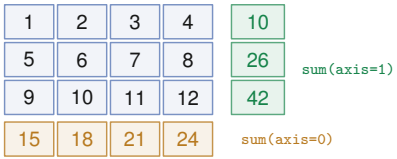
a.astype("int64")           # convert; drops decimals
np.arange(0, 10, 2)         # [0 2 4 6 8]
np.linspace(0, 1, 5)        # [0. 0.25 0.5 0.75 1.]
np.zeros((2, 3))            # 2 x 3 array of 0.0
```

**One dtype per array**

- Mixing ints and floats gives floats
- `np.array([1, "a"])` turns every element into text, which is why one text value spoils a numeric column
- `a[:, 1]` is *all rows, column 1*: the index before the comma picks rows, the one after picks columns

Slicing, masks and reshaping are the same ideas you will meet again in Pandas, with labels added.

# Axis and Broadcasting




- `axis=0` collapses the rows: one result per *column*. `axis=1` collapses the columns: one result per *row*
- **Broadcasting** combines arrays of different shapes: compare the shapes from the right; two sizes are compatible if they are equal or one of them is 1

*M = np.arange(1, 13).reshape(3, 4)*

```
shares = M / M.sum(axis=0)                    # (3,4) / (4,)  →  each entry as a
    share of its column
z = (M - M.mean(axis=0)) / M.std(axis=0)      # z-score of every column, no loop
```

The vector of four column sums is stretched down all three rows. When a shape error appears, print `.shape` of both operands first.

# Vectorized Thinking, and Why It Matters

## ✗ Loop, the habit from C

```
rates = []
for i in range(len(byte_counts)):
    if durations[i] > 0:
        rates.append(byte_counts[i]
                     / durations[i])
    else:
        rates.append(0.0)
```

## ✓✓ Vectorized

```
rates = np.where(durations > 0,
                 byte_counts / durations,
                 0.0)
```

- One expression, no index arithmetic
- Reads as the definition of the quantity
- The loop runs in the Python interpreter; the array expression runs in compiled code

| One million values, one machine | Loop | `np.where` |
|---|---|---|
| Time | 0.23 s | 0.006 s |

If you are writing `for i in range(len(...))` over a dataset, there is almost always a NumPy or Pandas expression that says the same thing. Notebook section 3 times it on your own runtime.

# Masks, `np.where` and Missing Values

```
d = np.array([3.4, 0.0, 61.5, 0.02])      # seconds
b = np.array([7047, 95, 22600, 60])       # bytes

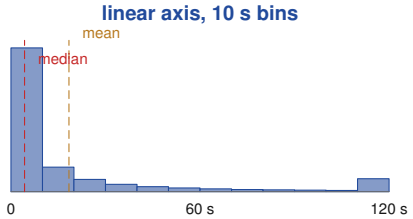
mask = d > 0              # [ True False  True  True]
mask.sum()                # 3      True counts as 1
mask.mean()               # 0.75   share of True
b[mask]                   # [7047 22600 60]

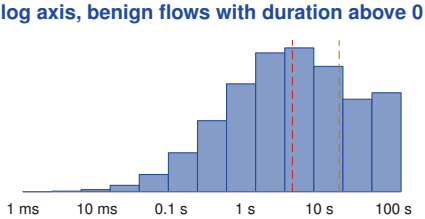
np.where(mask, b / d, 0.0)    # rate, or 0.0 if d == 0

np.mean([1.0, np.nan])        # nan  one NaN poisons it
np.nanmean([1.0, np.nan])     # 1.0  ignore the NaN
np.nan == np.nan              # False; use np.isnan(x)
```

- A comparison gives an array of booleans, and a boolean array can select, count and average
- `b / d` still divides by zero before `np.where` discards that entry, so NumPy may print a warning; the result is right
- NaN is contagious by design: it says “unknown”, and an unknown plus anything is unknown

# Skew, Logs and Why Means Mislead




- Benign flow duration: mean 18.4 s, median 4.3 s
- On a linear axis nearly everything is in the first bin; on a log axis the shape appears
- The spike at 120 s is a cap in this simulated table; real exporters also split long flows at a timeout
- `np.log10(0)` is undefined: the 152 zero-duration flows need `np.log1p` or a filter

A mean of a skewed quantity describes no typical flow. Report the median or quartiles beside it, and look at the distribution before you summarize it.

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# Anatomy of a DataFrame

columns, each a `Series` with one dtype

| index | duration | fwd_pkts | bwd_bytes | protocol | label |
|---|---|---|---|---|---|
| 0 | 4.12 | 9 | "1,842" | TCP | BENIGN |
| 1 | 0.03 | 2 | "0" | TCP | PortScan |
| 2 | 61.5 | 318 | "402,110" | UDP | BENIGN |
| dtype | float64 | int64 | object (text) | object | object |

one column is a `Series`; a table of columns is a `DataFrame`

in the raw file `bwd_bytes` is text, so its dtype is `object`, not `int64`

*Three rows of the course table. Seventy-four more columns are hidden to the right.*

- The **index** labels the rows; the **columns** label the fields; every column has exactly one **dtype**
- `object` is Pandas 2.x for text (Pandas 3 calls it `str`); a numeric column that shows `object` contains at least one value that is not a number

# Reading a CSV Without Surprises

```python
# defaults: Pandas infers every type from the values
df = pd.read_csv("flows.csv")

# "6,520" becomes the integer 6520
df = pd.read_csv("flows.csv", thousands=",")

# be explicit: fewer columns, fixed types, more NaN spellings
df = pd.read_csv(
    "flows.csv",
    usecols=["dst_port", "duration", "label"],
    dtype={"dst_port": "int32"},
    na_values=["unknown", "?"],
)

# a quick peek at a very large file
head = pd.read_csv("flows.csv", nrows=1000)
```

- Pandas decides each column's type by reading its values. One stray string makes the whole column text
- `thousands=","` solves the `bwd_bytes` problem at load time
- `usecols` keeps memory low when a file has hundreds of columns
- `nrows` lets you check a file's structure without waiting for all of it

Loading is where data quality is first decided. Choose types yourself when you know them, then print `df.dtypes`.

# CSV, Parquet and Where Files Live

| | CSV | Parquet |
|---|---|---|
| Layout | text, one row per line | binary, by column |
| Column types | not stored, inferred | stored in the file |
| Course table on disk | 50.4 MB | 43.2 MB |
| Read all 79 columns | about 0.8 s | about 0.14 s |
| Read 2 columns | about 0.27 s | about 0.01 s |
| Human readable | yes | no |

- A columnar file reads only the columns you ask for, and compresses each column on its own. The textbook reports 5 to 10 times less I/O than row formats for typical ML workloads
- Parquet remembers dtypes, so the `bwd_bytes` problem cannot recur once the file is written
- In Colab, `/content` is temporary and Drive is persistent: write results you need to Drive

CSV is the exchange format and forgets types. Once a table is cleaned, save it in a typed format.

Timings measured here (Pandas 3.0.2, PyArrow, one machine); expect other numbers on Colab. [MLS] §6.7.3 and §6.9, paraphrased

# Six Lines That Characterize Any New CSV

```
df = pd.read_csv("flows.csv")

df.shape                    # (rows, columns): how much data is there?
df.info()                   # dtypes and non-null counts: what is missing or mistyped?
df.describe()               # min / max / mean / quartiles: ranges and skew
df["label"].value_counts(normalize=True)    # class balance, before anything else
df.isna().sum().sort_values(ascending=False).head(10)     # worst columns
df.head(3)                  # sanity check: does a row look right?
```

- `info()` tells you what `describe()` cannot: which columns are text when they should be numbers
- `value_counts(normalize=True)` on the label is the single most important line on this slide

⚠ If the label distribution is 99.6% to 0.4%, every accuracy figure you compute for the rest of the semester needs that number printed beside it.

# Reading the Output

```
>>> df.shape
(112450, 79)

>>> df["label"].value_counts(normalize=True).
    round(4)
BENIGN      0.9963
PortScan    0.0037

>>> df.isna().sum().sort_values(ascending=False)
    .head(3)
flow_bytes_s    152
protocol          0
dst_port          0
```

**What you have just learned**

- 112 450 rows is small enough for Colab and large enough to be interesting
- The base rate is **0.37%**: the Session 1 accuracy paradox, in your own data
- The missing values sit in one derived rate column, and they are exactly the flows with zero duration

Three commands, and you already know the two facts that will shape every modeling decision: the class balance and where the data is broken.

# Reading `info()` and `describe()`

```
RangeIndex: 112450 entries, 0 to 112449
Data columns (total 79 columns):
 #  Column         Non-Null Count   Dtype
--- ------         --------------   -----
 0  dst_port       112450 non-null  int64
 1  protocol       112450 non-null  object
 2  duration       112450 non-null  float64
 3  fwd_pkts       112450 non-null  int64
 4  bwd_pkts       112450 non-null  int64
 5  fwd_bytes      112450 non-null  int64
 6  bwd_bytes      112450 non-null  object
 7  flow_bytes_s   112298 non-null  float64
...
78  label          112450 non-null  object
dtypes: float64(47), int64(29), object(3)
memory usage: 67.8+ MB
```

- Non-null below the row count means missing values (row 7)
- `object` on a numeric-looking column means text (row 6)

| | duration | fwd_pkts | fwd_bytes |
|---|---|---|---|
| mean | 18.33 | 10.82 | 1955.5 |
| std | 31.62 | 7.94 | 2917.4 |
| min | 0.00 | 1 | 0 |
| 25% | 1.05 | 5 | 467 |
| 50% | 4.27 | 8 | 1030 |
| 75% | 17.57 | 14 | 2241 |
| max | 120.00 | 54 | 67160 |

- Mean well above median: a right-skewed column
- `bwd_bytes` is missing from `describe()` because it is text

# Missing Values: Where They Come From

| Cause | Example in the notebook | Honest treatment |
|---|---|---|
| Undefined computation | `flow_bytes_s` when `duration` is 0 | keep NaN, or recompute with a stated rule |
| Failed conversion | `"unknown"` in a text column after `to_numeric(errors="coerce")` | count the coerced values before moving on |
| Never collected | a field the exporter did not fill | add an indicator column |

```
df["flow_bytes_s"].isna().sum()                          # 152
(df["duration"] == 0).sum()                              # 152: the same rows
df["flow_bytes_s_missing"] = df["flow_bytes_s"].isna()   # keep the fact that it was missing
```

Dropping, filling and flagging are all choices. Whichever you make must be a rule written in code, deterministic, with any statistic computed on the training data only.

[MLS] Ch. 6, §6.7.1, p. 363 (cleaning must be deterministic and reproducible; paraphrased)

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# Selection and Filtering

```
# column selection
dur = df["duration"]                          # a Series
sub = df[["duration", "fwd_pkts", "label"]]  # a DataFrame

# boolean masks: read them out loud
attacks   = df[df["label"] != "BENIGN"]
long_udp  = df[(df["protocol"] == "UDP") & (df["duration"] > 60)]
web_ports = df[df["dst_port"].isin([80, 443, 8080])]

# label-based and position-based access
df.loc[df["label"] == "PortScan", ["duration", "fwd_pkts"]]   # by name
df.iloc[0:5, 0:4]                                             # by position
```

- Combine masks with `&` and `|`, and parenthesize each condition: Python's `and` does not work element-wise
- Prefer `.loc` with names over `.iloc` with positions; column order changes, names do not

# Masks Step by Step

| duration | protocol | > 1 | = UDP | AND |
|---|---|---|---|---|
| 0.03 | TCP | False | False | False |
| 61.5 | UDP | True | True | True |
| 4.1 | TCP | True | False | False |
| 0.9 | UDP | False | True | False |
| 120.0 | UDP | True | True | True |

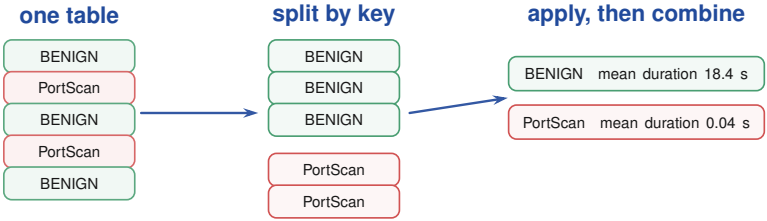
rows kept

```
m1 = df["duration"] > 1
m2 = df["protocol"] == "UDP"

df[m1 & m2]       # both
df[m1 | m2]       # either
df[~m1]           # not
```

- A mask has one boolean per row; indexing with it keeps the `True` rows
- `&`, `|` and `~` work element by element; each side needs its own parentheses when written inline
- `mask.sum()` counts the rows kept, `mask.mean()` gives their share

# Group, Apply, Combine




*`df.groupby("label")["duration"].mean()`: in one line you find that port scans are hundreds of times shorter than benign flows (0.04 s against 18.4 s).*

- **Split** the rows by the values of a key; **apply** a function to each group; **combine** the results into one table
- The key can be a column, several columns, or a computed expression

# What Grouping Reveals

```
cols = ["duration", "fwd_pkts", "fwd_bytes"]
df.groupby("label")[cols].agg(["mean", "median"]).round(2)
```

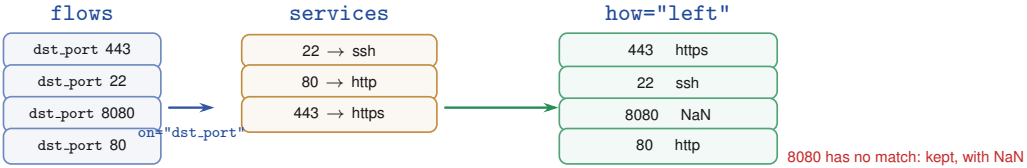
| | duration | | fwd_pkts | | fwd_bytes | |
|---|---|---|---|---|---|---|
| | mean | med. | mean | med. | mean | med. |
| BENIGN | 18.40 | 4.31 | 10.85 | 8 | 1962.8 | 1035 |
| PortScan | 0.04 | 0.04 | 1.42 | 1 | 0.0 | 0 |

- Port scans in this table are one to three packets, tens of milliseconds and no payload: they separate from benign flows on all three columns
- Within benign traffic, mean and median differ a lot; within port scans they agree
- `size()` counts rows, `count()` counts non-missing values, `nunique()` counts distinct ones

```
pd.crosstab(df["protocol"], df["label"])      # ICMP 1137 | 0, TCP 87380 | 416, UDP 23517 | 0
```

A per-class `groupby` is the fastest way to see which features can separate the classes at all. In this table every port scan is TCP, and that is a fact about the simulation, not about the world.

# Merge: Attaching Context




`left` keeps every flow (112 450 rows). `inner` keeps only matches (57 760). A duplicated key on the right repeats flows.

- `merge` matches rows by key, as a database join does; `how` decides what happens to rows without a match
- Always compare the row count before and after. A join that silently changes the size of your table changes your data

# Joining in Practice, and the Duplicate-Key Trap

```
services = pd.DataFrame({"dst_port": [22, 80, 443], "service": ["ssh", "http", "https"]})
n = len(df)                                        # 112450

m = df.merge(services, on="dst_port", how="left", validate="many_to_one")
assert len(m) == n                                 # a left join with unique keys keeps every flow
m["service"] = m["service"].fillna("other")        # 54690 flows on other ports

# what goes wrong when the lookup table has a duplicate key
bad = pd.DataFrame({"dst_port": [22, 80, 80, 443], "service": ["ssh", "http", "web", "https"]})
df.merge(bad, on="dst_port", how="left")           # 133058 rows: every port-80 flow appears twice
df.merge(bad, on="dst_port", how="left", validate="many_to_one")   # MergeError: keys are not unique
```

- `validate="many_to_one"` states the assumption about the lookup table, and Pandas checks it for you
- Fill the unmatched rows with an explicit value, and count them: they are the ports your lookup table does not know

# Deriving New Columns

```
df = df.assign(
    bytes_total   = df["fwd_bytes"] + df["bwd_bytes"],
    bytes_per_pkt = lambda d: d["bytes_total"] / (d["fwd_pkts"] + d["bwd_pkts"]),
    is_web        = df["dst_port"].isin([80, 443, 8080]),
    dur_class     = pd.cut(df["duration"], [-0.001, 0.1, 10, 121],
                           labels=["short", "medium", "long"]),
)

df["dur_class"].value_counts()          # how many flows fall in each band
df.sort_values("duration", ascending=False).head(3)
df.nlargest(3, "fwd_bytes")             # the three heaviest flows
```

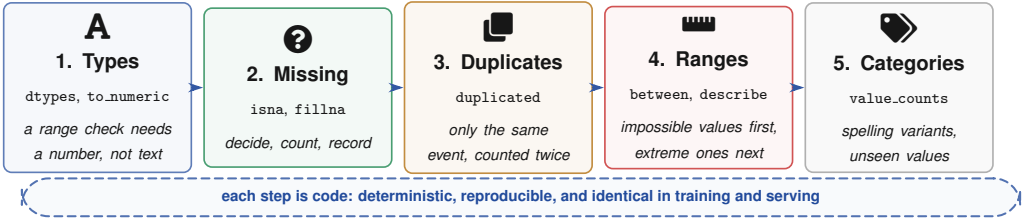
- `assign` adds columns without changing the original table, so a chain reads top to bottom
- A derived column has to be computed by a function you can call again on new data: it is the seed of the features you build later
- `bwd_bytes` must already be numeric here; the cleaning section shows how to make it so

Bands chosen by eye, like the ones above, are a modeling decision. Write down why you chose them.

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# A Cleaning Checklist




- The order matters: fix types first, because you cannot test the range of a column that is still text
- Print the shape, the missing counts and the dtypes after each step. That printout is your cleaning log
- Given the same input, a cleaning step must give the same output, whether it runs in training or in serving

[MLS] Ch. 6, §6.7.1, pp. 363–364: cleaning must be deterministic and reproducible (paraphrased)

# Fixing a Text Column

```python
# 1. at load time
df = pd.read_csv("flows.csv", thousands=",")

# 2. after loading: strip the separator, then convert
df["bwd_bytes"] = (df["bwd_bytes"]
                   .str.replace(",", "")
                   .astype("int64"))

# 3. tolerant: unparseable values become NaN
before = small["ttl"].isna().sum()
small["ttl"] = pd.to_numeric(small["ttl"], errors="coerce")
lost = small["ttl"].isna().sum() - before
print(lost, "values could not be parsed")

# check the result, every time
df["bwd_bytes"].dtype, df["bwd_bytes"].describe()
```

- Option 1 is shortest, but it only works when you know the separator in advance
- Option 3 comes from the notebook's 8-row table, where `ttl` contains the word `"unknown"`
- `errors="coerce"` turns problems into NaN without a warning, so always count what it converted

Option 2 works once; run it again and it fails, because the column is no longer text. Section 9 fixes that.

# Duplicates, Impossible Values and Outliers

```
df.duplicated().sum()                    # 0 exact duplicate rows here
df = df.drop_duplicates()                # only for the same event
       twice

(df["duration"] < 0).sum()               # impossible: 0 here
df["dst_port"].between(0, 65535).all()   # a valid port range

# 3-sigma rule, with statistics from the TRAINING data
mu, sd = train["duration"].mean(), train["duration"].std()
outlier = (df["duration"] - mu).abs() > 3 * sd
outlier.mean()                           # 0.057: 5.7% of flows
```

- The 3-sigma rule flags 6 452 flows here, and 6 092 of them are exactly the flows capped at 120 s
- Extreme values can be data errors or rare real events; a flood or a scan is an outlier by definition
- A threshold from the mean and standard deviation is consistent in serving only if those two numbers come from training and are stored

⚠ Do not delete extreme flows just because they are extreme. Decide what each one is, write the rule in code, and log how many rows it touched.

[MLS] Ch. 6, §6.7.1, p. 364: outlier handling and the consistency it requires (paraphrased)

# Encoding and Scaling: A Preview

```
# categories → indicator columns
pd.get_dummies(df["protocol"], prefix="proto")
#   proto_ICMP  proto_TCP  proto_UDP   (True / False)

# a numeric code that means a category → bands
pd.cut(df["dst_port"], [-1, 1023, 49151, 65535],
       labels=["well-known", "registered", "dynamic"])

# scaling with statistics from the training data only
mu, sd = train["duration"].mean(), train["duration"].std()
z = (df["duration"] - mu) / sd
vocab = sorted(train["protocol"].unique())   # keep it
```

- Encodings must handle the categories seen in training and the unseen ones at serving: keep the training vocabulary and map anything new to an “unknown” value
- Scaling parameters are computed once, on training data, and saved with the model. Port bands follow the standard ranges

You will use all of this in the preprocessing session. Today the habit matters: every transformation is a function with stored parameters.

[MLS] Ch. 6, §6.7.1, p. 365: normalization and categorical encoding with persisted parameters (paraphrased)

# From Raw File to Validated Table

```python
def clean(raw):
    df = raw.copy()
    df["bwd_bytes"] = pd.to_numeric(
        df["bwd_bytes"].astype(str).str.replace(",", ""))
    return df.drop_duplicates()

raw = pd.read_csv("flows.csv")
df = validate(clean(raw))          # loud if anything is off
```

| | raw | cleaned |
|---|---|---|
| rows and columns | 112450 × 79 | 112450 × 79 |
| text columns | 3 | 2 |
| integer columns | 29 | 30 |
| missing in `flow_bytes_s` | 152 | 152 |

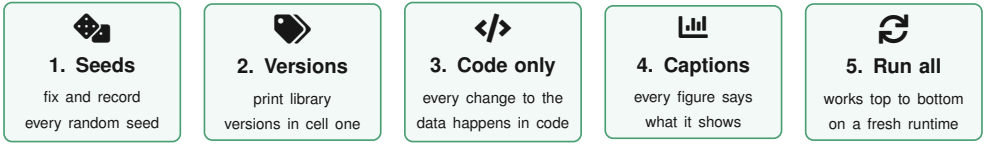
- One function takes the raw table to the cleaned one; the raw file is never edited
- `validate` from earlier passes on the output and would have failed on the input
- The 152 missing values are still there. Cleaning changed the type problem and left the missing-value question to a deliberate decision
- Running `clean` on an already clean table changes nothing

The pattern to keep: raw in, one function, validated out. Notebook section 7 asks you to write the type fix, and section 8 asks you to run everything again from the top.

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# Five Rules




- Rules 1 and 2 make the numbers repeatable; rule 3 makes the path from raw to clean data visible; rule 4 makes results readable; rule 5 tests all the others
- The notebook you open today already follows them, so you can copy its structure

These five rules are part of the A1 rubric for every notebook you submit this semester, including the Session 11 mini-lab.

# Why This Is Not Bureaucracy

- *A result you cannot reproduce is not a result.* In Session 8 you will deliberately break a notebook and repair it; without seeds you will not be able to tell whether your repair changed anything
- *Someone else will run your notebook.* In the mini-lab it is your group; in a workplace it is a colleague reading your incident analysis six months later
- *The notebook is the record of your decisions.* A markdown cell saying why a column was dropped is worth more than the line that dropped it
- *Silent divergence is expensive.* A different Pandas version can change a default and move a number, and you will never suspect the library

Write the notebook for the person who will read it without you in the room. Most of the time, that person is you in three months.

# Seeds and Deterministic Transformations

```
rng = np.random.default_rng(42)      # a generator you pass around
rng.normal(size=3)
# array([ 0.30471708, -1.03998411,  0.7504512 ])  every run

df.sample(frac=0.1, random_state=42)   # the same rows every run
np.random.seed(42)                     # older global style

# depends on the clock: the answer changes tomorrow
df["age_days"] = (pd.Timestamp.now() - df["start"]).dt.days

# depends on a recorded constant: the answer is stable
REF = pd.Timestamp("2026-09-21")
df["age_days"] = (REF - df["start"]).dt.days
```

- A deterministic transformation gives the same output for the same input, and does not depend on the time, on random numbers or on mutable global state
- Random steps use a seeded generator; the seed is a recorded parameter
- A “days since” feature computed from today’s date silently changes when you reprocess old data: store the reference date instead

[MLS] Ch. 6, §6.7.2, pp. 367–368: deterministic transformations, seeded randomness, explicit reference dates (paraphrased)

# Idempotent Cells: Switch, Not Toggle

```
# toggle 1: a second run divides again, with no error
df["duration"] = df["duration"] / 60
# run twice: 3.395 s → 0.0566 → 0.000943

# toggle 2: a second run fails
df["bwd_bytes"] = (df["bwd_bytes"].str.replace(",", "")
                   .astype("int64"))
# AttributeError: Can only use .str accessor with string values

# switch: any number of runs gives the same table
df["duration_min"] = raw["duration"] / 60
df["bwd_bytes"] = pd.to_numeric(
    df["bwd_bytes"].astype(str).str.replace(",", ""))
```

- A light switch set to “on” twice is still on; a toggle changes state every press. Data steps should behave like the switch
- Idempotent steps can be retried after a failure without duplicating or corrupting anything
- In a notebook, “retry” means “I ran that cell again”, and you do that all the time
- The safe pattern: derive new columns from the raw table, and never overwrite a column with a function of itself

[MLS] Ch. 6, §6.7.2, pp. 366–367: idempotency and the light-switch analogy (paraphrased)

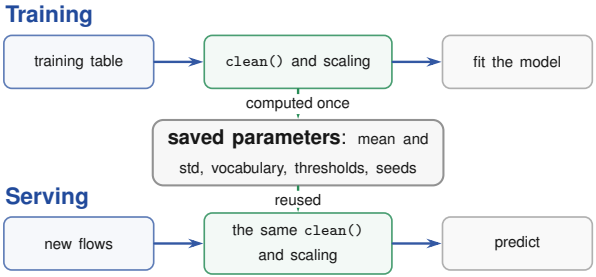
# Lineage: Recording What You Did

| Textbook practice | What it tracks | Your notebook version |
|---|---|---|
| Transformation versioning | which code produced which dataset, and the library versions | a version line in cell one; the printed `pd.__version__` |
| Parameter tracking | constants used: means, standard deviations, vocabularies | named constants defined once and printed |
| Processing lineage | raw files read, steps applied, in which order | the raw file stays untouched; every step is a cell; a text cell says why |
| Code version | the commit that produced the result | the notebook in version control, saved with its outputs |

Lineage answers one question: if a prediction is wrong, can I recreate exactly the data the model saw? Without it, a bug in the pipeline cannot be fixed safely.

[MLS] Ch. 6, §6.7.4, pp. 370–371 (paraphrased; the notebook column is our translation)

# Training and Serving Must Agree




**If serving re-implements the cleaning**, the model receives inputs prepared differently from its training data: **training-serving skew**

- A small case: training code strips the comma from `"6,520"`, serving code does not, and the model gets text where it expected a number
- The textbook cites production studies that put inconsistent processing behind a large share of ML failures, and names feature stores as the industrial fix

[MLS] Ch. 6, §6.4.1, pp. 338–339 and §6.7.1, pp. 363–365 (paraphrased)

# Outline

1. The Colab Environment and the Notebook Contract
2. Data Before Models
3. The Data: Network Flow Tables
4. Python Essentials for Data Work
5. NumPy: Arrays, Shapes and Vectorized Thinking
6. Pandas: Loading and Inspecting a Dataset
7. Selecting, Filtering, Grouping and Joining
8. Cleaning a Table in Code
9. Reproducibility Rules for This Course
10. Checkpoint, Activity and Summary

# Checkpoint: Predict the Output

```python
df = pd.DataFrame({"port": [22, 80, 22, 443], "bytes": [100, 250, 90, 500]})

# 1
print(df.shape)

# 2
print(df[df["port"] == 22]["bytes"].mean())

# 3
print(df.groupby("port")["bytes"].sum().loc[22])
```

## And two questions in words

- What does `df.info()` tell you that `df.describe()` does not?
- Why does a notebook that only works when cells are run out of order fail the reproducibility rule?

Notebook section 6: write your prediction in the cell first, then run it. Predicting before running is the habit that finds the gaps.

# Checkpoint: Answers and Reasoning

1. `(4, 2)`: four rows, two columns. `shape` is rows first
2. `95.0`: the mask keeps the two rows with port 22, whose bytes are 100 and 90, and their mean is 95.0
3. `190`: `groupby` sums the bytes per port; port 22 has 100 + 90

- `info()` reports each column's dtype and its non-null count. `describe()` summarizes only the numeric columns, so a text column that should be numeric simply disappears from it
- A notebook that needs an unusual order depends on hidden state in the kernel. Nobody else can reproduce it, and neither can you after a restart

If a prediction was wrong, do not just read the answer. Find the line of reasoning that failed: shape order, mask logic, or what `groupby` returns.

# In-Class Activity: Guided Data Inspection

**Alone or in pairs, on the course flow table. Twenty minutes.** Answer each question *with code*, not by scrolling.

1. How many rows and how many columns?
2. What is the class balance, as a proportion?
3. What are the five most frequent destination ports?
4. What is the mean flow duration for each class?
5. How many missing values are there per column, worst first?
6. Name one column whose dtype is wrong, say what it should be, and then fix it

**In the notebook: section 7**

```
check("Q4", your_answer)
# Q4: correct.
#   or
# Q4: not quite. Re-read the
# question, then check units
# and rounding.
```

- Each check says right or wrong and never reveals the answer
- The check accepts any equivalent form: a Series, a dict or a tuple
- `report()` lists what is still open
- Finish with two sentences describing the dataset in your own words

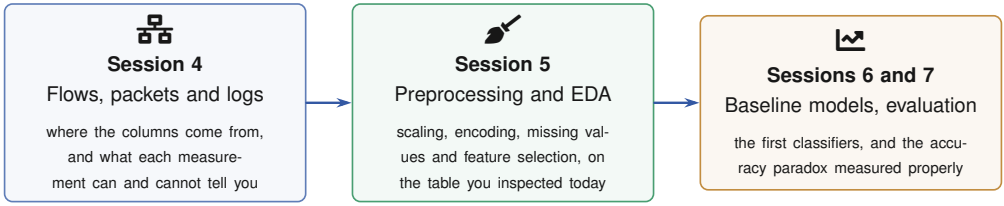
The solutions are revealed at minute 80. Compare your code, not only your answers: there are usually three reasonable ways to write each one.

# Summary

- Colab is a temporary machine with the libraries pre-installed; work that must survive goes to Drive. A notebook must run top to bottom on a fresh runtime
- In ML, data defines behavior and data problems fail silently. Quality has several dimensions, and each has its own check
- A flow table has instances, features and a label. The dtype is not the meaning, and the label balance is the first thing to look at
- NumPy gives shape discipline and vectorized expressions; six Pandas lines characterize any new CSV
- `groupby` shows which features separate the classes; `merge` attaches context, and you check the row count afterwards
- Cleaning is code: deterministic, idempotent, with parameters computed on training data and recorded. The five reproducibility rules apply to every notebook

Next session: where these columns come from (packets, flows, PCAP files, logs) and what that lets you conclude.

# Where This Goes Next




- Everything in those sessions starts with a table you understand. Today's inspection routine is the entry ticket
- Bring the notebook, with its output, to the next session

# Self-Study Questions

1. Why does a data quality problem in an ML system rarely produce an error message? Give one example from the flow table
2. Name the four pillars of data engineering and one notebook practice that serves each
3. What is a data cascade, and why is the collection stage the most expensive place for an error?
4. A model that always predicts `BENIGN` scores 99.63% accuracy on the course table. What is its recall for `PortScan`?
5. Why is `dst_port` a poor numeric feature as stored, and what would you do instead?
6. Write a NumPy expression, with no loop, that gives every entry of a 2-D array as a share of its column sum
7. What does `validate="many_to_one"` do in a merge? What happens to the row count if you leave it out and the lookup table has a duplicate key?
8. Why does `df["duration"] = df["duration"] / 60` break the reproducibility rules? Rewrite it so that it does not
9. Give one example of training-serving skew that could arise with network flow data

Answers are in the slides above. Write yours in a markdown cell at the end of the notebook: it is good practice for the written part of the exam.

# Homework and Reading

## **Homework** (assessed under A1)

- Open the lab notebook, save a copy to your Drive, and complete every “Your turn” cell and the six activity questions
- Write two sentences describing the dataset in your own words, then run the reproducibility checks of section 8 until `report()` shows nothing open
- Submit the `.ipynb` executed end to end on a fresh runtime. The stretch tasks S1 to S3 are optional

## Reading

- **[MLS]** Ch. 6: §6.1 to §6.4 (pp. 324–344) and §6.7 (pp. 362–371)
- **[G&S]** Ch. 1: §4.2 to §5.2; Ch. 8: §3
- **[Géron]** Ch. 2, data loading and quick-look sections; supplementary: the official “10 minutes to pandas” guide

Bring one column you do not understand to Session 4; we will trace it back to the packets.

Lab notebook (Colab):
`link.uit.edu.vn/NT120-Wk3`

Course page:
`link.uit.edu.vn/NT120`

# Appendix: Pandas Reference, Inspect and Select

| Task | Call |
|---|---|
| Size and shape | `df.shape`, `len(df)` |
| Types and non-null counts | `df.info()`, `df.dtypes` |
| Summary statistics | `df.describe()`, `df["c"].describe()` |
| First rows, random rows | `df.head()`, `df.sample(5, random_state=42)` |
| Class balance | `df["label"].value_counts(normalize=True)` |
| Missing values | `df.isna().sum()`, `df.isna().mean()` |
| Duplicates and distinct values | `df.duplicated().sum()`, `df["c"].nunique()`, `df["c"].unique()` |
| One column, several columns | `df["c"]`, `df[["a", "b"]]` |
| Filter rows | `df[df["c"] > 0]`, `df.query("c > 0")`, `df["c"].isin([1, 2])` |
| By name, by position | `df.loc[mask, ["a", "b"]]`, `df.iloc[0:5, 0:3]` |

# Appendix: Pandas Reference, Transform, Group, Combine, Save

| Task | Call |
|---|---|
| Convert a type | `pd.to_numeric(s, errors="coerce")`, `s.astype("int64")` |
| Clean text | `s.str.replace(",", "")`, `s.str.lower()`, `s.str.strip()` |
| Fill or drop missing | `df.fillna(0)`, `df.dropna(subset=["c"])` |
| New columns | `df.assign(x=...)`, `np.where(cond, a, b)` |
| Bands | `pd.cut(s, bins, labels=...)`, `pd.qcut(s, 4)` |
| Group and aggregate | `df.groupby("k")["c"].agg(["mean", "median", "size"])` |
| Cross tabulate | `pd.crosstab(a, b)`, `df.pivot_table(index=..., columns=..., values=..., aggfunc="mean")` |
| Join tables | `df.merge(other, on="k", how="left", validate="many_to_one")` |
| Sort and rank | `df.sort_values("c", ascending=False)`, `df.nlargest(3, "c")` |
| Save | `df.to_csv(path, index=False)`, `df.to_parquet(path)` |

# Appendix: NumPy Reference

| Task | Call |
|---|---|
| Create | `np.array(...)`, `np.zeros((r, c))`, `np.arange(a, b, step)`, `np.linspace(a, b, n)` |
| Inspect | `a.shape`, `a.ndim`, `a.dtype`, `a.size` |
| Reshape and convert | `a.reshape(3, 4)`, `a.T`, `a.astype("float64")` |
| Index and slice | `a[0, 2]`, `a[:, 1]`, `a[a[:, 0] > 1]` |
| Reduce along an axis | `a.sum(axis=0)`, `a.mean(axis=1)`, `a.std(axis=0)` |
| Conditions | `np.where(mask, x, y)`, `mask.sum()`, `np.any(mask)`, `np.all(mask)` |
| Missing values | `np.isnan(a)`, `np.nanmean(a)`, `np.nan_to_num(a)` |
| Logs | `np.log10(a)`, `np.log1p(a)` (defined at 0) |
| Random numbers | `rng = np.random.default_rng(42)`, `rng.normal(size=n)` |

# Appendix: Colab Troubleshooting

| Symptom | Likely cause | What to do |
|---|---|---|
| `NameError` after reconnecting | the runtime restarted, so variables are gone | Runtime, Run all |
| `FileNotFoundError` | the file was on a discarded runtime, or the path is wrong | upload again or mount Drive; `!ls` |
| `ModuleNotFoundError` | the library is not installed | `!pip install name`, then restart if asked |
| Different numbers from a classmate | another seed or library version | print the versions, fix the seed |
| A cell never finishes | a very slow loop | Runtime, Interrupt execution |
| Your edits are lost | you edited the shared original | File, Save a copy in Drive |
| `object` where a friend sees `str` | Pandas 2 and Pandas 3 name text dtypes differently | not an error |

# Appendix: Glossary

**Cell**: a block of code or text in a notebook.

**Kernel**: the Python process that runs the cells and holds the variables.

**Runtime**: the virtual machine behind a Colab notebook.

**DataFrame, Series**: a labeled table, and one column of it.

**dtype**: how a column's values are stored.

**NaN**: the marker for a missing numeric value.

**Mask**: a boolean array used to select rows.

**Broadcasting**: NumPy's rule for combining arrays of different shapes.

**Flow**: the packets sharing a 5-tuple within a time window, summarized as one record.

**Instance, feature, label**: a row, an input column, and the column to predict.

**Data cascade**: downstream failures caused by an early data quality problem.

**Idempotent**: giving the same result however many times it runs.

**Lineage**: the recorded history from raw data to final features.

**Training-serving skew**: features computed differently in training and in production.

**Seed**: the starting value that makes random numbers repeatable.

# Appendix: Where the Theory Comes From

| Slide topic | Source | Where |
|---|---|---|
| Data defines behavior, silent failure | [MLS] | Ch. 6, §6.1, pp. 324–325 |
| Where the time goes; four pillars | [MLS] | §6.2, Fig. 6.1, 6.2, Table 6.1, pp. 326–329 |
| Data cascades | [MLS] | §6.3, Fig. 6.3, p. 330 |
| Pipeline layers, validation checks, skew | [MLS] | §6.4, Fig. 6.5, pp. 335–339 |
| Cleaning, transformation, encoding | [MLS] | §6.7.1, pp. 363–365 |
| Idempotent and deterministic processing | [MLS] | §6.7.2, pp. 366–368 |
| Lineage and versioning | [MLS] | §6.7.4, pp. 370–371 |
| Parquet and columnar storage | [MLS] | §6.7.3 (footnote), §6.9 |
| Flow records and flow features | [G&S] | Ch. 1, §4.2 and Table 1 |
| CTU dataset, class imbalance | [G&S] | Ch. 1, §5.1 and §5.2 |
| Relevant and redundant features | [G&S] | Ch. 8, §3 |

[MLS] V. J. Reddi, *Machine Learning Systems*, online textbook (October 2025 PDF), printed page numbers. [G&S] B. B. Gupta and M. Sheng (eds.), *Machine Learning for Computer and Cyber Security*, CRC Press. [Géron] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow.* Textbook content is paraphrased; figures are redrawn. The dataset facts were computed on the notebook's simulated table.